DATA STRUCTURE LAB

LABORATORY MANUAL

MCA, Semester -III

Subject Code: E1PY203B

Session: 2024-25, Even Semester

GALGOTIAS UNIVERSITY
SCHOOL OF COMPUTER APPLICATIONS & TECHNOLOGY

Table of Contents

1. Vision and Mission of the Institute
2. Vision and Mission of the Department
3. Programme Educational Objectives (PEOs)
4. Programme Outcomes (POs)
5. Programme Specific Outcomes (PSOs)
6. University Syllabus
7. Course Outcomes (COs)
8. CO- PO and CO-PSO mapping
9. Course Overview
10. List of Experiments
11. DOs and DON'Ts
12. General Safety Precautions
13. Guidelines for students for report preparation
14. Lab assessment criteria
15. Details of Conducted Experiments
16. Lab Experiments

Vision and Mission of the University

Vision

To be recognized globally as a premier School of Computer Applications and Technology imparting quality and value based education engaged in multi-disciplinary and collaborative research.

Mission

The mission of the department is to :

M1: Developing a strong foundation in fundamentals of computer applications with responsiveness towards emerging technologies.

M2: Establishing state-of-the-art facilities and adopt education 4.0 practices to analyze, develop, test and deploy sustainable ethical IT solutions by involving multiple stakeholders.

M3: Establishing Centers of Excellence for multidisciplinary collaborative research in association with industry and academia**.**

PROGRAM EDUCATIONAL OBJECTIVES

The Post Graduates of Computer Application shall:

PEO1: be able to successfully pursue research in Computer Applications and allied disciplines at institutions of transnational reputation.

PEO2: serve in technical or managerial roles at Government firms, Corporates and contributing to the society as successful entrepreneurs through startup.

PEO3: be engaged with leading Global Software services companies, Consultancy organizations and academic Institutions.

PROGRAMME SPECIFIC OUTCOME (PSO):

The students of Computer Application shall:

PSO1: Have the ability to work with contemporary technologies in computing requisite to Industry 4.0 developing and implementing solutions to real life problems.

PSO2: Demonstrate applications development skills learned through technical training and projects to solve real world problems.

PROGRAMME OUTCOME (PO):

1. Computational Knowledge: Apply knowledge of computing fundamentals, computing specialization, mathematics, and domain knowledge appropriate for the computing specialization to the abstraction and conceptualization of computing models from defined problems and requirements.

2. Problem Analysis: Identify, formulate, research literature, and solve complex computing problems reaching substantiated conclusions using fundamental principles of mathematics, computing sciences, and relevant domain disciplines.

3. Design /Development of Solutions: Design and evaluate solutions for complex computing problems, and design and evaluate systems, components, or processes that meet specified needs with appropriate consideration for public health and safety, cultural, societal, and environmental considerations.

4. Conduct investigations of complex Computing problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern Tool Usage: Create, select, adapt and apply appropriate techniques, resources, and modern computing tools to complex computing activities, with an understanding of the limitations.

6. Professional Ethics: Understand and commit to professional ethics and cyber regulations, responsibilities, and norms of professional computing practices.

7. Life-long Learning: Recognize the need, and have the ability, to engage in independent learning for continual development as a computing professional.

8. Project management and finance: Demonstrate knowledge and understanding of the computing and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

9. Communication Efficacy: Communicate effectively with the computing community, and with society at large, about complex computing activities by being able to comprehend and write

effective reports, design documentation, make effective presentations, and give and understand clear instructions.

10. Societal and Environmental Concern: Understand and assess societal, environmental, health, safety, legal, and cultural issues within local and global contexts, and the consequential responsibilities relevant to professional computing practices.

11. Individual and Team Work: Function effectively as an individual and as a member or leader in diverse teams and in multidisciplinary environments.

12. Innovation and Entrepreneurship: Identify a timely opportunity and using innovation to pursue that opportunity to create value and wealth for the betterment of the individual and society at large.

University Syllabus

COURSE CONTENT : THEORY+ PRACTICAL

THEORY	**Module 1**	**Arrays and Stack:**	**8 hours**
	Introduction Basic Terminology, Elementary Data Organization, Data Structure operations, Abstract Data Type (ADT) - Algorithm Complexity and Time-Space trade-off, Efficiency of an Algorithm, Asymptotic notations: Big Oh, Big Theta and Big Omega. **Arrays**: Array Definition, Representation, and Analysis - Single and Multidimensional Arrays- Row Major Order, and Column Major Order, **Application of Arrays:** Sparse Matrices and their representations. **Stack:** Representation and Implementation of Stack (Array and Linked list) - Operations on Stack – push and pop Application of Stack: Conversion of Infix to Prefix and Postfix Expressions - Evaluation of postfix expression – Recursion. Principles of recursion, Tail recursion, Problem solving using recursion with examples such as Tower of Hanoi.		
	Module 2	**Queue & Link Lists:**	**8 hours**
	Queue: Representation and Implementation of Queue (Array and Linked list) - Create, Add, Delete, Full and Empty Circular Queue – Dequeue – Priority Queue. **Lists**: Representation and Implementation of Singly Linked Lists - Doubly Linked Lists – Circular Linked Lists, Operations on a Linked List. Insertion, Deletion, Traversal Application of Lists: Polynomial Representation and Addition.		
	Module 3	**Trees:**	**8 hours**
	Binary Trees: Basic Terminology - Binary Tree representation - Complete Binary Tree - Extended Binary Trees - Binary Trees Traversals-: Inorder, Preorder and Postorder **Binary Search Trees**: Binary Search Tree Operation of Insertion, Deletion, Searching & Modification of data in Binary Search Tree, AVL Trees, B-Trees.		
	Module 4	**Graphs**:	**8 hours**
	Graphs: Basic Terminology and Representations – Graphs (Adjacency Matrices, Adjacency List) & Multi-Graphs - Directed Graphs - Adjacency Matrices - Graph Traversals (Depth First Search and Breadth First Search) - Spanning Trees - Minimum Cost Spanning Trees- Prims and Kruskal algorithm.		
	Module 5	**Searching, Sorting and File Structures:**	**8 hours**
	Searching: Sequential Search - Binary Search - **Sorting**: Insertion Sort, Bubble Sorting, Quick Sort, Merge Sort Concept of Hashing & Collision: Hash Table, Hash Functions, Collision Resolution Strategies, Hash Table Implementation, Linear probing, Quadratic, Double hashing.		

List of Experiments mapped with COs

Lab No	Points To Covered	Co Mapping	Blooms Level
1	Write a program to implement the following operation on Array: (a) Average (b) Insertion (c) Deletion (d) Searching (LS&BS) (e) Display (f) Reverse (g) Merging	CO1	L2
2	Write a program to implement STACK and its operations using array: a) Create b) Underflow c) Overflow d) PUSH e) POP	CO1	L2
3	1). Write a program to implement postfix evaluation using stack. 2) Recursion Example (Tower of Hanoi.	CO1	L2
4	Write a program to implement Queue and its operations using array: f) Create g) Underflow h) Overflow i) Enqueue j) Dequeue	CO1	L2
5	Write a program to implement DQueue and its operations using array: a) Create b) Underflow c) Overflow d) Enqueue e) Dequeue	CO1	L2

6	Write a program to implement Circular Queue and its operations using array: a) Create b) Underflow c) Overflow d) Enqueue e) Dequeue	CO1	L2
7	Write a program to implement the following in a singly linked list (a) Insertion at beginning (b) Insertion at end (c) Insert at defined position (d) Deletion from defined position.	CO1	L2
8	Write a program to implement the following in a Doubly linked list (a) Insertion at beginning (b) Insertion at end (c) Insert at defined position (d) Deletion from defined position.	CO1	L2
9	Write a program to implement Binary Search Tree and its operations.	CO1	L2
10	Implementation of Searching using Hash table.	CO2	L2
11	Implementation of Heap Sort	CO2	L2
12	Implementation of Quick sort, bubble sort, insertion sort and Merge sort	CO2	L2
13	Write a program to implement stack using an array and linked list	CO2	L2
14	Write a program to implement a graph using the array.	CO2	L2
15	Write a program to implement a graph traversal.	CO2	L2

Course Outcomes (COs)

- **COURSE OUTCOMES(COs) After the completion of the course, students will be able to:**

CO No.	Course Outcomes
E1PY203B.1	Apply fundamental concepts of arrays, linked lists, stacks, and queues to solve computational problems efficiently.
E1PY203B.2	Analyze different data structures such as trees and graphs, evaluating their efficiency for various operations like searching, sorting, and traversal.
E1PY203B.3	Develop algorithms for data structure operations, including stack-based expression evaluation, recursion-based problem-solving, and hashing techniques.
E1PY203B.4	Implement various searching and sorting techniques, including binary search, quick sort, and merge sort, along with file structures and hashing strategies.

- **BLOOM'S LEVEL OF THE COURSE OUTCOMES**

CO No.	Remember **KL1**	Understand **KL 2**	Apply **KL 3**	Analyse **KL 4**	Evaluate **KL 2**	Create **KL 6**
E1PY203B.1			√			
E1PY203B.2				√		
E1PY203B.3			√			√
E1PY203B.4			√			

Course Overview

This laboratory course covers fundamental and advanced data structures, focusing on their implementation, operations, and real-world applications.

Key Topics:

- **Introduction to Data Structures:** Basic terminology, operations, ADT, algorithm complexity, asymptotic notations (Big O, Θ, Ω).
- **Arrays:** Representation (Single & Multi-dimensional), Row & Column Major Order, Sparse Matrices.
- **Stacks:** Implementation using Arrays and Linked Lists, Push/Pop operations, Applications (Infix to Postfix/Prefix conversion, Postfix Evaluation, Recursion - Tower of Hanoi).
- **Queues:** Implementation using Arrays and Linked Lists, Circular Queue, Dequeue, Priority Queue.
- **Linked Lists:** Singly, Doubly, and Circular Linked Lists, Operations (Insertion, Deletion, Traversal), Applications (Polynomial Addition).
- **Trees:** Binary Trees, Complete & Extended Trees, Tree Traversals (Inorder, Preorder, Postorder), Binary Search Trees (BST), AVL Trees, B-Trees.
- **Graphs:** Representation (Adjacency Matrix & List), Directed & Undirected Graphs, Graph Traversals (BFS, DFS), Spanning Trees, Minimum Cost Spanning Trees (Prim's & Kruskal's Algorithm).
- **Searching & Sorting:** Sequential & Binary Search, Sorting Algorithms (Insertion Sort, Bubble Sort, Quick Sort, Merge Sort).
- **Hashing & File Structures:** Hash Tables, Hash Functions, Collision Resolution Strategies (Linear Probing, Quadratic, Double Hashing).

DOs and DON'Ts

DOs

1. Login-on with your username and password.
2. Log off the Computer every time when you leave the Lab.
3. Arrange your chair properly when you are leaving the lab.
4. Put your bags in the designated area.
5. Ask permission to print.

DON'Ts

1. Do not share your username and password.
2. Do not remove or disconnect cables or hardware parts.
3. Do not personalize the computer setting.
4. Do not run programs that continue to execute after you log off.
5. Do not download or install any programs, games or music on computer in Lab.
6. Personal Internet use chat room for Instant Messaging (IM) and Sites is strictly prohibited.
7. No Internet gaming activities allowed.
8. Tea, Coffee, Water & Eatables are not allowed in the Computer Lab.

General Safety Precautions

Precautions (In case of Injury or Electric Shock)

1. To break the victim with live electric source, use an insulator such as fire wood or plastic to break the contact. Do not touch the victim with bare hands to avoid the risk of electrifying yourself.
2. Unplug the risk of faulty equipment. If main circuit breaker is accessible, turn the circuit off.
3. If the victim is unconscious, start resuscitation immediately, use your hands to press the chest in and out to continue breathing function. Use mouth-to-mouth resuscitation if necessary.
4. Immediately call medical emergency and security. Remember! Time is critical; be best.

Ambulance : 9810611477(Fortis Ambulance)
120-2400222(Fortis Ambulance)

Precautions (In case of Fire)

2. Turn the equipment off. If power switch is not immediately accessible, take plug off.
3. If fire continues, try to curb the fire if possible by using the fire extinguisher or by covering it with a heavy cloth if possible isolate the burning equipment from the other surrounding equipment.
4. Sound the fire alarm by activating the nearest alarm switch located in the hallway.
5. Call security and emergency department immediately:

Emergency : 219 (Reception)
298(Health Center)

Guidelines to students for report preparation

All students are required to maintain a record of the experiments conducted by them. Guidelines for its preparation are as follows:-

1) All files must contain a title page followed by an index page. ***The files will not be signed by the faculty without an entry in the index page.***

2) Student's Name, Roll number and date of conduction of experiment must be written on all pages.

3) For each experiment, the record must contain the following

(i) Aim/Objective of the experiment

(ii) Pre-experiment work (as given by the faculty)

(iii) Lab assignment questions and their solutions

(iv) Test Cases (if applicable to the course)

(v) Results/ output

Note:

1. Students must bring their lab record along with them whenever they come for the lab.
2. Students must ensure that their lab record is regularly evaluated.

Lab Assessment Criteria

An estimated 10 lab classes are conducted in a semester for each lab course. These lab classes are assessed continuously. Each lab experiment is evaluated based on 5 assessment criteria as shown in following table. Assessed performance in each experiment is used to compute CO attainment as well as internal marks in the lab course.

Grading Criteria	Exemplary (4)	Competent (3)	Needs Improvement (2)	Poor (1)
Pre-Lab written work (this may be assessed through viva)	Complete procedure with underlined concept is properly written	Underlined concept is written but procedure is incomplete	Not able to write concept and procedure	Underlined concept is not clearly understood
Program Writing/ Modeling	Assigned problem is properly analyzed, correct solution designed, appropriate language constructs/ tools are applied, Program/solution written is readable	Assigned problem is properly analyzed, correct solution designed, appropriate language constructs/ tools are applied	Assigned problem is properly analyzed & correct solution designed	Assigned problem is properly analyzed
Identification & Removal of errors/ bugs	Able to identify errors/ bugs and remove them	Able to identify errors/ bugs and remove them with little bit of guidance	Is dependent totally on someone for identification of errors/ bugs and their removal	Unable to understand the reason for errors/ bugs even after they are explicitly pointed out
Execution & Demonstration	All variants of input /output are tested, Solution is well demonstrated and implemented concept is clearly explained	All variants of input /output are not tested, However, solution is well demonstrated and implemented concept is clearly explained	Only few variants of input /output are tested, Solution is well demonstrated but implemented concept is not clearly explained	Solution is not well demonstrated and implemented concept is not clearly explained
Lab Record Assessment	All assigned problems are well recorded with objective, design constructs and solution along with Performance analysis using all variants of input and output	More than 70 % of the assigned problems are well recorded with objective, design contracts and solution along with Performance analysis is done with all variants of input and output	Less than 70 % of the assigned problems are well recorded with objective, design contracts and solution along with Performance analysis is done with all variants of input and output	Less than 40 % of the assigned problems are well recorded with objective, design contracts and solution along with Performance analysis is done with all variants of input and output

LAB EXPERIMENTS

LAB EXPERIMENT 1

Lab Assessment Criteria

- **Understanding of Concepts**: 20%
- **Code Implementation & Accuracy**: 30%
- **Algorithm & Explanation**: 20%
- **Execution & Output**: 20%
- **Documentation & Presentation**: 10%

Experiment 1: Array Operations

Aim

To implement basic operations on arrays including average calculation, insertion, deletion, searching (Linear & Binary), display, reversing, and merging.

Learning Outcome

- Understand and manipulate arrays.
- Perform searching, insertion, deletion, and merging operations.
- Gain hands-on experience with fundamental array operations in C.

Algorithm

1. Declare and initialize the array.
2. Implement functions for the following:
   - **Average Calculation**: Iterate through elements and compute the mean.
   - **Insertion**: Shift elements and insert at the desired position.
   - **Deletion**: Remove element and shift remaining elements.
   - **Searching**: Implement Linear and Binary Search.
   - **Display**: Print array elements.
   - **Reverse**: Swap elements from start to end.
   - **Merging**: Combine two arrays sequentially.
3. Display the results after each operation.

C Language Code

```
#include <stdio.h>
#define SIZE 100

void display(int arr[], int n) {
   for (int i = 0; i < n; i++) {
      printf("%d ", arr[i]);
   }
   printf("\n");
}

void insert(int arr[], int *n, int pos, int val) {
   if (*n >= SIZE) {
      printf("Array Overflow\n");
      return;
   }
   for (int i = *n; i > pos; i--) {
```

```
        arr[i] = arr[i - 1];
    }
    arr[pos] = val;
    (*n)++;
}

void delete(int arr[], int *n, int pos) {
    if (*n == 0) {
        printf("Array Underflow\n");
        return;
    }
    for (int i = pos; i < *n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    (*n)--;
}

int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) return i;
    }
    return -1;
}

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (arr[mid] == key) return mid;
        else if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

void reverse(int arr[], int n) {
    for (int i = 0, j = n - 1; i < j; i++, j--) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}

int main() {
    int arr[SIZE], n, choice, val, pos;
    printf("Enter number of elements: ");
```

```
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    while (1) {
        printf("\n1. Insert\n2. Delete\n3. Search (Linear)\n4. Search (Binary)\n5. Reverse\n6.
Display\n7. Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter position and value: ");
                scanf("%d %d", &pos, &val);
                insert(arr, &n, pos, val);
                break;
            case 2:
                printf("Enter position: ");
                scanf("%d", &pos);
                delete(arr, &n, pos);
                break;
            case 3:
                printf("Enter value to search: ");
                scanf("%d", &val);
                pos = linearSearch(arr, n, val);
                printf(pos == -1 ? "Not Found\n" : "Found at %d\n", pos);
                break;
            case 4:
                printf("Enter value to search: ");
                scanf("%d", &val);
                pos = binarySearch(arr, n, val);
                printf(pos == -1 ? "Not Found\n" : "Found at %d\n", pos);
                break;
            case 5:
                reverse(arr, n);
                break;
            case 6:
                display(arr, n);
                break;
            case 7:
                return 0;
            default:
                printf("Invalid Choice!\n");
        }
    }
}
```

Experiment 2: Stack Operations Using Array

Lab Assessment Criteria:

- Understanding of stack operations (10 Marks)
- Implementation accuracy (10 Marks)
- Code efficiency and use of switch-case (5 Marks)
- Proper input and output handling (5 Marks)

Aim:

To implement stack operations (Create, Underflow, Overflow, Push, Pop) using an array and switch-case.

Learning Outcomes:

- Understand the concept of stack (LIFO principle).
- Implement stack operations using an array.
- Handle underflow and overflow conditions.

Algorithm:

1. **Initialize** an empty stack and define the maximum size.
2. **Display Menu** with the following operations:
   - **Push**: Insert an element into the stack.
   - **Pop**: Remove an element from the stack.
   - **Check Overflow**: Ensure the stack doesn't exceed the maximum size.
   - **Check Underflow**: Ensure the stack isn't empty before popping.
   - **Display Stack**: Show current elements.
3. **Use switch-case** to handle user choices and execute operations.
4. Repeat until the user exits.

C Program (Using Switch-Case):

STACK

```
#include<stdio.h>
#include<conio.h>
#define MAX 20

typedef struct stack
{
int data[MAX];
int top;
}stack;

void init(stack*);
int empty(stack*);
int full(stack*);
void push(stack*, int);
int pop(stack*);
void peak(stack*);

void main()
{
stack s;
```

```
int x, n;
clrscr();
init(&s);
printf("Enter 1 for PUSH\n");
printf("Enter 2 for POP\n");
printf("Enter 3 for PEAK ELEMENT\n");
printf("Enter 4 for EXIT\n");
printf("Enter for Choice\n");
scanf("%d",&x);
while(x<4)
{
        switch(x)
        {       case 1:
                        if(!full(&s))
                        {
                                printf("Enter the push element");
                                scanf("%d",&n);
                                push(&s,n);
                        }
                        else
                        {
                                printf("\n stack OVERFLOW");
                                exit(0);
                        }
                        break;
                case 2:
                        if(!empty(&s))
                        {
                                x=pop(&s);
                                printf("Deleted Element=%d\n", x);
                        }
                        break;
                case 3:
                        peak(&s);
                        break;
                default:
                        printf("DEFAULT");
                        break;
                }
                printf("Enter for Re choice");
                scanf("%d",&x);
}
    //  getch();
}//main ends

void init(stack *s)
{
s->top=-1;
}

int empty(stack *s)
```

```
{
if(s->top==-1)
{
        return(1);
}
return(0);
}
int full(stack *s)
{
if(s->top==MAX-1)
{
        return(1);
}
return(0);
}
void push(stack *s, int p)
{
s->top=s->top+1;
s->data[s->top]=p;
}
int pop(stack *s)
{
int x;
x=s->data[s->top];
s->top=s->top-1;
return(x);
}
void peak(stack *s)
{     int i;
if(s->top==-1)
        printf("Stack is empty\n");
else
    printf("Peak Element of STACK=%d\n",s->data[s->top]);

}
```

Explanation:

- The **stack is implemented using an array** of size MAX.
- **Push function** inserts elements at the top.
- **Pop function** removes the top element if the stack is not empty.
- **Overflow check** ensures elements do not exceed the maximum size.
- **Underflow check** ensures elements are present before popping.
- **Switch-case handles all user inputs** and executes the respective stack operation.

Experiment 3: Postfix Evaluation & Tower of Hanoi

Lab Assessment Criteria:

- Understanding of postfix evaluation using a stack (10 Marks)
- Implementation of recursion (Tower of Hanoi) (10 Marks)
- Code efficiency and use of switch-case (5 Marks)
- Proper input and output handling (5 Marks)

Aim:

1. To evaluate a **postfix expression** using a **stack**.
2. To implement **Tower of Hanoi** using **recursion**.

Learning Outcomes:

- Understand the concept of **postfix notation** and how to evaluate it using a **stack**.
- Learn **recursion** and apply it to solve **Tower of Hanoi**.
- Use **switch-case** for structured implementation.

- Part 1: Postfix Expression Evaluation

Algorithm:

1. Create an empty stack.
2. Read the postfix expression from left to right.
3. If the symbol is an operand, push it onto the stack.
4. If the symbol is an operator:
    - Pop the top two operands.
    - Perform the operation.
    - Push the result back onto the stack.
5. Repeat until the expression is fully evaluated.
6. The final result will be at the top of the stack.

C Code for Postfix Evaluation (Using Switch-Case):

Evaluation of postfix Expression

```
#include<stdio.h>
#include<conio.h>
#define MAX 20
typedef struct stack
{
int data[MAX];
int top;
}stack;

void init(stack*);
int empty(stack*);
int full(stack*);
void push(stack*, int);
int pop(stack*);
void peak(stack*);
int eval(char x, int op1, int op2);
```

```
void main()
{
stack s;
int x,n, op1, op2, val;
int i=0;
char pofx[50], x1;
clrscr();
init(&s);
printf("Enter 1 for PUSH\n");
printf("Enter 2 for POP\n");
printf("Enter 3 for PEAK ELEMENT\n");
printf("Enter 4 for EVALUATION of POSTFIX STRING\n");
printf("Enter 5 for EXIT\n");
printf("Enter for Choice\n");
scanf("%d",&x);
while(x<5)
{
        switch(x)
        {       case 1:
                        if(!full(&s))
                        {
                                printf("Enter the push element");
                                scanf("%d",&n);
                                push(&s,n);
                        }
                        else
                        {
                                printf("\n stack OVERFLOW");
                                exit(0);
                        }
                        break;
                case 2:
                        if(!empty(&s))
                        {
                                x=pop(&s);
                                printf("Deleted Element=%d\n", x);
                        }
                        else
                                printf("Stack is EMPTY\n");
                        break;
                case 3:
                        peak(&s);
                        break;
                case 4:
                    printf("\n Enter the expression\n");
                    scanf("%s",&pofx);
                    while((x1=pofx[i++])!='\0')
                    {
                                if(isdigit(x1))
```

```
                                        push(&s, x1-'0');
                                else
                                {
                                        op2=pop(&s);
                                        op1=pop(&s);
                                        val=eval(x1, op1, op2);
                                        push(&s, val);
                                }
                        }
                        val=pop(&s);
                        printf("\nvalue of expression= %d\n", val);
                        break;
                default:
                        printf("DEFAULT");
                        break;
                }
                printf("Enter for Re choice");
                scanf("%d",&x);
}
        //      getch();
}

void init(stack *s)
{
s->top=-1;
}

int empty(stack *s)
{
if(s->top==-1)
{
        return(1);
}
return(0);
}
int full(stack *s)
{
if(s->top==MAX-1)
{
        return(1);
}
return(0);
}
void push(stack *s, int p)
{
s->top=s->top+1;
s->data[s->top]=p;
}
int pop(stack *s)
{
int x;
```

```
x=s->data[s->top];
s->top=s->top-1;
return(x);
}
void peak(stack *s)
{
if(s->top==-1)
        printf("Stack is empty\n");
else
    printf("Peak Element of STACK=%d\n",s->data[s->top]);
    printf("TOP Value=%d\n", s->top);
}

int eval(char x, int op1, int op2)
{
if(x=='+')
        return(op1+op2);

if(x=='-')
        return(op1-op2);

if(x=='*')
        return(op1*op2);

if(x=='/')
        return(op1/op2);

if(x=='%')
        return(op1%op2);
}
}
```

Sample Output for Postfix Evaluation:

Enter a valid postfix expression: 53+82-*
Postfix Evaluation Result: 40

Explanation:

Expression: **5 3 + 8 2 - ***

1. 5 + 3 = 8
2. 8 - 2 = 6
3. 8 * 6 = 40

Part 2: Tower of Hanoi (Using Recursion)

Algorithm:

1. If only one disk, move it from **source** to **destination**.
2. Move n-1 disks from **source** to **auxiliary** using **destination** as helper.
3. Move the nth disk from **source** to **destination**.
4. Move n-1 disks from **auxiliary** to **destination** using **source** as helper.

C Code for Tower of Hanoi:

```
#include <stdio.h>

// Recursive function for Tower of Hanoi
void towerOfHanoi(int n, char source, char auxiliary, char destination) {
    if (n == 1) {
        printf("Move disk 1 from %c to %c\n", source, destination);
        return;
    }
    towerOfHanoi(n - 1, source, destination, auxiliary);
    printf("Move disk %d from %c to %c\n", n, source, destination);
    towerOfHanoi(n - 1, auxiliary, source, destination);
}

int main() {
    int n;

    printf("Enter number of disks: ");
    scanf("%d", &n);

    towerOfHanoi(n, 'A', 'B', 'C');

    return 0;
}
```

Sample Output for Tower of Hanoi (n=3):

pgsql
CopyEdit

```
Enter number of disks: 3
Move disk 1 from A to C
Move disk 2 from A to B
Move disk 1 from C to B
Move disk 3 from A to C
Move disk 1 from B to A
Move disk 2 from B to C
Move disk 1 from A to C
```

Explanation:

For $n = 3$, the sequence of moves follows the recursive steps:

1. Move **disk 1** from A → C
2. Move **disk 2** from A → B
3. Move **disk 1** from C → B
4. Move **disk 3** from A → C
5. Move **disk 1** from B → A
6. Move **disk 2** from B → C
7. Move **disk 1** from A → C

Conclusion:

- **Postfix Evaluation** was implemented using a **stack and switch-case**.
- **Tower of Hanoi** was solved using **recursion**.

• Experiment 4: Queue Operations Using Array

Lab Assessment Criteria:

- Understanding of queue operations (10 Marks)
- Implementation accuracy (10 Marks)
- Code efficiency and use of switch-case (5 Marks)
- Proper input and output handling (5 Marks)

Aim:

To implement **queue operations** (Create, Underflow, Overflow, Enqueue, Dequeue) using an **array** and **switch-case** in C.

Learning Outcomes:

- Understand the **FIFO (First-In-First-Out)** principle of queues.
- Implement **basic queue operations** using an **array**.
- Handle **underflow** and **overflow** conditions effectively.

Algorithm:

1. Initialize an **empty queue** with front = -1 and rear = -1.
2. **Display Menu** with options:
   - **Enqueue (Insert Element)**
   - **Dequeue (Remove Element)**
   - **Check Underflow Condition**
   - **Check Overflow Condition**
   - **Display Queue Elements**
3. Use **switch-case** to handle user input and execute the chosen operation.
4. Repeat until the user **exits**.

C Code (Queue Using Array with Switch-Case):

```
#include<stdio.h>
#include<conio.h>
#define MAX 5

typedef struct Q
{
int R, F;
int data[MAX];
}Q;

void create(Q*);
int enqueue(Q*,int); //exit() not working
int dequeue(Q*);
void Print(Q*);

void main()
{
    Q P;  int x, i,n,m;
clrscr();
```

- A queue is a linear list of elements in which deletions can take place only at one end, called front.
- And insertions can take place only at the other end, called the rear.
- Queue is also called FIFO (first in first out)
- Rear:- keeps the status of last added item in queue.
- front:- keeps the status of first item of queue.

```
create(&P);
printf("1-PUSH, 2-POP, 3-PRINT, 4-EXIT\n");
printf("Enter Your Choice\n");
scanf("%d",&n);
while(n<4)
{
        switch(n)
        {
                case 1:
                        printf("Enter the value\n");
                        scanf("%d", &x);
                        enqueue(&P, x);
                        break;
                case 2:
                    m=dequeue(&P);
                    printf("Deleted element%d\n",m);
                        break;
                case 3:
                        Print(&P);
                        break;
                default :
                printf("exit");
                break;
        }
        printf("Enter Your Choice\n");
        scanf("%d",&n);
}//while ends
}
```

void create(Q *P)

```
{
P->R=-1;
P->F=-1;
}
```

int enqueue(Q *P, int x)

```
{
if(P->R==MAX-1)
{
        printf("Queue is FULL\n");
        return;
}
if(P->R==-1)
{
        P->R=P->F=0;
        P->data[P->R]=x;
return;
}
else
        P->R=P->R+1;
```

```
        P->data[P->R]=x;
        return;
}

int dequeue(Q *P)
{
int x;
x=P->data[P->F];
if(P->F==-1)
{
        printf("UNDERFLOW\n");
    //  exit(0);
}
if(P->R==P->F)
{
        P->R=P->F=-1;
}
else
{
        P->F=P->F+1;
}
        return(x);
}

void Print(Q *P)
{
int i;
if(P->F==-1)
{
        printf("Queue is EMPTY\n");
}
for(i=P->F; i<=P->R; i++)
{
        printf("%d\n", P->data[i]);
}
}
```

Explanation:

- **Queue is implemented using an array** of size MAX.
- **Enqueue function** adds elements to the **rear**.
- **Dequeue function** removes elements from the **front**.
- **Overflow check** prevents inserting beyond MAX.
- **Underflow check** prevents removing from an empty queue.
- **Switch-case manages all user operations dynamically**.

- **Experiment 5: Double-Ended Queue (DQueue) Operations Using Array**

Lab Assessment Criteria:

- Understanding of **DQueue operations** (10 Marks)
- Implementation accuracy (10 Marks)
- Code efficiency and use of **switch-case** (5 Marks)
- Proper input and output handling (5 Marks)

Aim:

To implement **Double-Ended Queue (DQueue) operations** (Create, Underflow, Overflow, Enqueue, Dequeue) using an **array** and **switch-case** in C.

Learning Outcomes:

- Understand the **Double-Ended Queue (DQueue)** concept.
- Implement **Insertion and Deletion** from both **Front and Rear**.
- Handle **Underflow** and **Overflow** conditions.

Algorithm:

1. **Initialize** front = -1 and rear = -1 to represent an **empty DQueue**.
2. **Display Menu** with options:
   - **Insert at Front**
   - **Insert at Rear**
   - **Delete from Front**
   - **Delete from Rear**
   - **Check Underflow Condition**
   - **Check Overflow Condition**
   - **Display Queue Elements**
3. **Use switch-case** to handle user input and execute the chosen operation.
4. Repeat until the user **exits**.

C Code (DQueue Using Array with Switch-Case):

- Experiment 6: Circular Queue Operations Using Array

Lab Assessment Criteria:

- Understanding of **Circular Queue operations** (10 Marks)
- Implementation accuracy (10 Marks)
- Code efficiency and use of **switch-case** (5 Marks)
- Proper input and output handling (5 Marks)

Aim:

To implement **Circular Queue operations** (Create, Underflow, Overflow, Enqueue, Dequeue) using an **array** and **switch-case** in C.

Learning Outcomes:

- Understand the **concept of Circular Queue** and its advantage over linear queues.
- Implement **Insertion (Enqueue) and Deletion (Dequeue)** in a **circular manner**.
- Handle **Underflow** and **Overflow** conditions.

Algorithm:

1. **Initialize** front = -1 and rear = -1 to represent an **empty Circular Queue**.
2. **Display Menu** with options:
   - **Enqueue (Insert at Rear)**
   - **Dequeue (Remove from Front)**
   - **Check Underflow Condition**
   - **Check Overflow Condition**
   - **Display Queue Elements**
3. **Use switch-case** to handle user input and execute the chosen operation.
4. Repeat until the user **exits**.

C Code (Circular Queue Using Array with Switch-Case):

D-Queue

```
#include<stdio.h>
#include<conio.h>
#define MAX 20

typedef struct Q
{
int R, F;
int data[MAX];
}Q;

void create(Q*);
void enqueue(Q*,int);
int dequeue(Q*);
int dqdelete_rear(Q*);
void Print(Q*);
void main()
{
    Q P;
int x, i, n ,m, a;

clrscr();
create(&P);
printf("1-PUSH, 2-POP, 3-PRINT, 4-dqdelete_rear, 5-EXIT\n");
printf("Enter Your Choice\n");
scanf("%d",&n);
while(n<5)
{
        switch(n)
        {
                case 1:
                        printf("Enter the value\n");
                        scanf("%d", &x);
                        enqueue(&P, x);
                        break;
                case 2:
                    m=dequeue(&P);
```

```
                    printf("Deleted element%d\n",m);
                        break;
                case 3:
                        Print(&P);
                        break;

                case 4:
                        a=dqdelete_rear(&P);
                    printf("Deleted element%d\n",a);
                        break;

                default :
                printf("exit");
                break;
        }
        printf("\nEnter Your Choice\n");
        scanf("%d",&n);
}//while ends
}

void create(Q *P)
{
P->R=-1;
P->F=-1;
}

void enqueue(Q *P, int x)
{
if(P->R==MAX-1)
{
        printf("Queue is FULL\n");
        exit(0);
}
if(P->R==-1)
{
        P->R=P->F=0;
        P->data[P->R]=x;
}
else
        P->R=P->R+1;
        P->data[P->R]=x;
}

int dequeue(Q *P)
{
int x;
x=P->data[P->F];
if(P->F==-1)
{
        printf("UNDERFLOW\n");
    //  exit(0);
```

```
}
if(P->R==P->F)
{
	P->R=P->R=-1;
}
else
{
	P->F=P->F+1;
}
	return(x);
}


int dqdelete_rear( Q *P)
{
int x;
if(P->R==-1)
{
	printf("queue is empty\n");
	//exit(0);
}
else
{
	x = P->data[P->R];
	P->R=P->R-1;
}
return(x);
}


void Print(Q *P)
{
int i;
if(P->F==-1)
{
	printf("Queue is EMPTY\n");
}
for(i=P->F; i<=P->R; i++)
{
	printf("%d\n", P->data[i]);
}
}
```

Experiment 7: Singly Linked List Operations

Lab Assessment Criteria:

- Understanding of **Singly Linked List operations** (10 Marks)
- Implementation accuracy (10 Marks)
- Code efficiency and use of **switch-case** (5 Marks)
- Proper input and output handling (5 Marks)

Aim:

To implement the following **Singly Linked List** operations using **switch-case** in C:

1. **Insertion at the beginning**
2. **Insertion at the end**
3. **Insertion at a defined position**
4. **Deletion from a defined position**

Learning Outcomes:

- Understand the **dynamic memory allocation** used in linked lists.
- Implement **insertions and deletions** at different positions.
- Use **switch-case** for structured implementation.

Algorithm:

1. **Create a linked list** with a struct Node containing data and next pointer.
2. **Display Menu** with options:
    - **Insert at Beginning**
    - **Insert at End**
    - **Insert at a Defined Position**
    - **Delete from a Defined Position**
    - **Display Linked List**
3. **Use switch-case** to handle user input and execute the chosen operation.
4. Repeat until the user **exits**.

C Code (Singly Linked List with Switch-Case):

INSERTION IN LINKED LIST

```
#include<stdio.h>
#include<conio.h>

typedef struct node
{
int data;
struct node *next;
}node;

node *create(int);
node *addbeg(node*);
node *insertmid(node*);
node *insertend(node*);
void Print(node*);

void main()
{
node *start; int n,x;
```

```
clrscr();
start=NULL;
printf("Enter the no. of node\n");
scanf("%d",&n);
start=create(n);
Print(start);
Printf("1-Beg, 2-MID, 3-Last");
printf("Enter your Choice\n");
scanf("%d", &x);
while(x<5)
{       switch(x)
{
        case 1:
            start=addbeg(start);
        break;
        case 2:
                start=insertmid(start);
        break;
        case 3:
                start=insertend(start);
        break;
        case 4:
        Print(start);
        break;
        default:
        printf("enter 5 for quit");
        break;
}
        printf("Enter for rechoice\n");
        scanf("%d",&x);

}
getch();
}

node *create(int n)
{
node *head, *p, *temp;
int i;
head=(node*)malloc(sizeof(node));
head->next=NULL;
printf("Enter the value");
scanf("%d", &head->data);
p=head;
for(i=1; i<n; i++)
{
        temp=(node*)malloc(sizeof(node));
        p->next=temp;
        p=p->next;
        printf("enter the first onwords values\n");
        scanf("%d", &p->data);
```

```
        p->next=NULL;
}
return(head);
}

void Print(node *p)
{
while(p!=NULL)
{
        printf("->%d", p->data);
        printf("\n");
        p=p->next;
}
}
node *addbeg(node *p)
{
node *tmp, *head; int item;
head=p;
tmp=(node*)malloc(sizeof(node));
printf("Enter the FIRST value\n");
scanf("%d",&item);
tmp->data=item;
tmp->next=head;
head=tmp;
return(head);
}
node *insertend(node *head)
{
node *p, *q; int item;
p=(node*)malloc(sizeof(node));
printf("Enter the LAST value");
scanf("%d",&item);
p->data=item;
p->next=NULL;
if(head==NULL)
        return(p);
q=head;
while(q->next!=NULL)
{
        q=q->next;
}
q->next=p;
return(head);
}
node *insertmid(node *head)
{
node *p, *q; int item, i, loc;
p=(node*)malloc(sizeof(node));
printf("Enter the LOCATION of item\n");
scanf("%d", &loc);
printf("Enter the item value\n");
```

```
scanf("%d", &item);
p->data=item;
p->next=NULL;
q=head;
for(i=0; i<loc-1; i++)
{
        q=q->next;

        if(q==NULL)
        {
                printf("there are less then element\n");
                return(head);
        }
}
p->next=q->next;
q->next=p;
return(head);
}
```

Explanation:
Singly Linked List is implemented dynamically using struct Node.
Insert at Beginning: Adds a node at the head of the list.
Insert at End: Traverses the list and adds a node at the tail.
Insert at Position: Traverses to the specified position and inserts the node.
Delete from Position: Removes a node from the given position.
Switch-case dynamically manages user operations.
Sample Output:
mathematica
Copy
Edit
Menu:
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Position
5. Display Linked List
6. Exit
Enter your choice: 1
Enter value to insert at beginning: 10
10 inserted at the beginning.

Enter your choice: 2
Enter value to insert at end: 20
20 inserted at the end.

Enter your choice: 3
Enter value and position to insert: 15 2
15 inserted at position 2.

Enter your choice: 5
Linked List: 10 -> 15 -> 20 -> NULL

Enter your choice: 4
Enter position to delete: 2
Node with value 15 deleted from position 2.

Enter your choice: 5
Linked List: 10 -> 20 -> NULL

Enter your choice: 6
Exiting program...

Conclusion:

- **Implemented Singly Linked List operations** (Insertion at Beginning, End, Position & Deletion).
- **Used switch-case** for structured and interactive implementation.
- **Handled memory allocation dynamically using malloc()**

Experiment 8: Doubly Linked List Operations

Lab Assessment Criteria:

- Understanding of **Doubly Linked List operations** (10 Marks)
- Implementation accuracy (10 Marks)
- Code efficiency and use of **switch-case** (5 Marks)
- Proper input and output handling (5 Marks)

Aim:

To implement the following **Doubly Linked List** operations using **switch-case** in C:

1. **Insertion at the beginning**
2. **Insertion at the end**
3. **Insertion at a defined position**
4. **Deletion from a defined position**

Learning Outcomes:

- Understand **Doubly Linked List structure** (bidirectional navigation).
- Implement **Insertions and Deletions** at different positions.
- Use **switch-case** for structured implementation.

Algorithm:

1. **Create a doubly linked list** with a struct Node containing data, prev, and next pointers.
2. **Display Menu** with options:
    - **Insert at Beginning**
    - **Insert at End**
    - **Insert at a Defined Position**
    - **Delete from a Defined Position**
    - **Display Linked List**
3. **Use switch-case** to handle user input and execute the chosen operation.
4. Repeat until the user **exits**.

C Code (Doubly Linked List with Switch-Case):

```
#include <stdio.h>
#include <stdlib.h>

// Structure for a node in a doubly linked list
struct Node {
   int data;
   struct Node* prev;
   struct Node* next;
};

struct Node* head = NULL;  // Initialize empty doubly linked list
```

```
// Function to insert a node at the beginning
void insertAtBeginning(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = value;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL)
        head->prev = newNode;

    head = newNode;
    printf("%d inserted at the beginning.\n", value);
}

// Function to insert a node at the end
void insertAtEnd(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
    } else {
        struct Node* temp = head;
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
        newNode->prev = temp;
    }
    printf("%d inserted at the end.\n", value);
}

// Function to insert a node at a defined position
void insertAtPosition(int value, int position) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory allocation failed!\n");
        return;
    }
    newNode->data = value;
```

```
    if (position == 1) {
        newNode->prev = NULL;
        newNode->next = head;
        if (head != NULL)
            head->prev = newNode;
        head = newNode;
    } else {
        struct Node* temp = head;
        for (int i = 1; temp != NULL && i < position - 1; i++)
            temp = temp->next;
        if (temp == NULL) {
            printf("Invalid position!\n");
            free(newNode);
            return;
        }
        newNode->next = temp->next;
        newNode->prev = temp;
        if (temp->next != NULL)
            temp->next->prev = newNode;
        temp->next = newNode;
    }
    printf("%d inserted at position %d.\n", value, position);
}

// Function to delete a node from a defined position
void deleteFromPosition(int position) {
    if (head == NULL) {
        printf("List is empty! Nothing to delete.\n");
        return;
    }

    struct Node* temp = head;

    if (position == 1) {
        head = head->next;
        if (head != NULL)
            head->prev = NULL;
        printf("Node with value %d deleted from position %d.\n", temp->data, position);
        free(temp);
        return;
    }

    for (int i = 1; temp != NULL && i < position; i++)
        temp = temp->next;

    if (temp == NULL) {
        printf("Invalid position! Nothing deleted.\n");
        return;
    }

    if (temp->next != NULL)
```

```
        temp->next->prev = temp->prev;

    if (temp->prev != NULL)
        temp->prev->next = temp->next;

    printf("Node with value %d deleted from position %d.\n", temp->data, position);
    free(temp);
}

// Function to display the linked list
void display() {
    if (head == NULL) {
        printf("Doubly Linked List is empty.\n");
        return;
    }
    struct Node* temp = head;
    printf("Doubly Linked List: NULL <-> ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    int choice, value, position;

    do {
        printf("\nMenu:\n");
        printf("1. Insert at Beginning\n2. Insert at End\n3. Insert at Position\n4. Delete from Position\n5.
Display Linked List\n6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert at beginning: ");
                scanf("%d", &value);
                insertAtBeginning(value);
                break;
            case 2:
                printf("Enter value to insert at end: ");
                scanf("%d", &value);
                insertAtEnd(value);
                break;
            case 3:
                printf("Enter value and position to insert: ");
                scanf("%d %d", &value, &position);
                insertAtPosition(value, position);
                break;
```

```
        case 4:
            printf("Enter position to delete: ");
            scanf("%d", &position);
            deleteFromPosition(position);
            break;
        case 5:
            display();
            break;
        case 6:
            printf("Exiting program...\n");
            break;
        default:
            printf("Invalid choice! Try again.\n");
    }
  } while (choice != 6);

  return 0;
}
```

Explanation:

- **Doubly Linked List is implemented dynamically using struct Node**.
- **Insert at Beginning**: Adds a node at the head of the list.
- **Insert at End**: Traverses the list and adds a node at the tail.
- **Insert at Position**: Traverses to the specified position and inserts the node.
- **Delete from Position**: Removes a node from the given position.
- **Switch-case dynamically manages user operations**.

Experiment 9: Binary Search Tree (BST) Operations

Aim:

To implement **Binary Search Tree (BST) operations** using **switch-case** in C.

Operations Implemented:

- **Insertion**
- **Deletion**
- **Searching**
- **Inorder, Preorder, Postorder Traversal**

C Code (Binary Search Tree with Switch-Case):

Program showing various operations on binary search tree

```
#include<conio.h>
#include<stdio.h>
#include<stdlib.h>

typedef struct BSTnode
{
 int data;
 struct BSTnode *left,*right;
}BSTnode;

BSTnode *find(BSTnode *, int);
BSTnode *insert(BSTnode *, int);
BSTnode *delet(BSTnode *, int);
BSTnode *create();
void inorder(BSTnode * T);
void preorder(BSTnode * T);
void postorder(BSTnode * T);

void main()
{
BSTnode *root=NULL,*p;
int x, op;
clrscr();
do
  { printf("\n\n1)Create\n2)Delete\n3)Search\n4)Preorder");
    printf("\n5)Inorder\n6)Posorder\n7)Insert\n8)Quit");
```

```
    printf("\nEnter Your Choice :");
    scanf("%d",&op);
     switch(op)
      {
            case 1: root=create(); break;
            case 2:  printf("\nEnter the key to be deleted :");
                     scanf("%d", &x);
                     root=delet(root, x);
                     break;
            case 3:  printf("\nEnter the key to be searched :");
                     scanf("%d", &x);
                     p=find(root, x);
                     if(p==NULL)
                       printf("\n ***** Not Found****");
                     else
                       printf("\n ***** Found*****");
                     break;
            case 4: preorder(root);break;
            case 5: inorder(root);break;
            case 6: postorder(root);break;
            case 7: printf("\nEnter a Data : ");
                     scanf("%d",&x);
                     root=insert(root, x);
                     break;
      }
 }while(op!=8);
}

void inorder(BSTnode *T)
{
if(T!=NULL)
{
            inorder(T->left);
            printf("%d\t", T->data);
            inorder(T->right);
}
}

void preorder(BSTnode *T)
 {          if(T!=NULL)
  { printf("%d\t",T->data);
    preorder(T->left);
    preorder(T->right);
  }
 }
void postorder(BSTnode *T)
 {          if(T!=NULL)
  {
    postorder(T->left);
    postorder(T->right);
    printf("%d\t",T->data);
```

```
  }
 }
```

BSTnode *find(BSTnode *root, int x)

```
{       BSTnode *p;
p=root;
while(p!=NULL)
{
        if(x==p->data)
                return(p);
        if(x>p->data)
                p=p->right;
        else
        p=p->left;
}
return(p);
}
```

BSTnode *insert(BSTnode *T, int x)

```
{
BSTnode *p,*q,*r;
// acquire memory for the new node
r=(BSTnode*)malloc(sizeof(BSTnode));
r->data=x;
r->left=NULL;
r->right=NULL;
if(T==NULL)
        return(r);
// find the leaf node for insertion
p=T;
while(p!=NULL)
{
        q=p;
        /*if(x==p->data)
          {
            printf("\nDuplicate Data : ");
            return(T);
          }*/
        if(x>=p->data)
                p=p->right;
        else
                p=p->left;
}
if(x>=q->data)
        q->right=r;  // x as right child of q
else
        q->left=r;   //x as left child of q
```

```
return(T);
}
```

BSTnode *create()

```
{
int n,x,i;
BSTnode *root;
root=NULL;
printf("\nEnter no. of nodes :");
scanf("%d", &n);
for(i=0;i<n; i++)
{
        printf("\nEnter tree values %d:", i);
scanf("%d", &x);
        root=insert(root, x);
}
return(root);
}
```

BSTnode *delet(BSTnode *T, int x)

```
{
BSTnode *temp;
if(T==NULL)
{
        printf("\nElement not found :");
        return(T);
}
if(x < T->data)             // delete in left subtree
{
        T->left=delet(T->left, x);
        return(T);
}
if(x >T->data)              // delete in right subtree
{
        T->right=delet(T->right, x);
        return(T);
}
// element is found
if(T->left==NULL && T->right==NULL)   // a leaf node
{
        temp=T;
        free(temp);
        return(NULL);
}
if(T->left==NULL)
{
        temp=T;
        T=T->right;
        free(temp);
```

```
        return(T);
}
if(T->right==NULL)
{
        temp=T;
        T=T->left;
        free(temp);
        return(T);
}
// node with two children
//go to the inorder successor of the node
temp =T->right;
while(temp->left !=NULL)
temp=temp->left;
T->data=temp->data;
T->right=delet(T->right, temp->data);
return(T);
}
```

Sample Output:

mathematica
CopyEdit

```
Menu:
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Exit
Enter your choice: 1
Enter value to insert: 50
50 inserted.

Enter your choice: 1
Enter value to insert: 30
30 inserted.

Enter your choice: 1
Enter value to insert: 70
70 inserted.

Enter your choice: 4
Inorder Traversal: 30 50 70

Enter your choice: 2
Enter value to delete: 30
30 deleted.

Enter your choice: 4
Inorder Traversal: 50 70

Enter your choice: 3
```

```
Enter value to search: 70
Value found in BST.

Enter your choice: 7
Exiting program...
```

Conclusion:

- **Implemented Binary Search Tree operations** (Insertion, Deletion, Search, Traversals).
- **Used switch-case** for structured and interactive implementation.
- **Handled dynamic memory allocation using malloc()**.

Explanation:

- **BST is implemented dynamically using struct Node**.
- **Insert function** inserts values at the correct position based on BST rules.
- **Delete function** removes a node while maintaining BST properties.
- **Search function** finds a value in the BST.
- **Traversal functions (Inorder, Preorder, Postorder) are implemented recursively**.
- **Switch-case dynamically manages user operations**.

Experiment 10: Searching Using Hash Table

Lab Assessment Criteria:

- Understanding of **Hash Table operations** (10 Marks)
- Implementation accuracy (10 Marks)
- Code efficiency and use of **switch-case** (5 Marks)
- Proper input and output handling (5 Marks)

Aim:

To implement **searching using Hash Table** in C with **collision handling** using **Linear Probing**.

Learning Outcomes:

- Understand **hashing concepts** and the role of **hash functions**.
- Implement **Insertion and Searching** in a **Hash Table**.
- Handle **collisions using Linear Probing**.

Algorithm:

1. **Initialize a Hash Table** of size MAX with all values set to -1 (empty).
2. **Display Menu** with options:
   - **Insert a value** using a hash function.
   - **Search for a value** in the Hash Table.
   - **Display the Hash Table**.
3. **Use switch-case** to handle user input and execute the chosen operation.
4. **Handle collisions using Linear Probing** if the computed index is occupied.
5. Repeat until the user **exits**.

C Code (Hash Table with Linear Probing and Switch-Case):

Hashing with Separate Chaining

```
#include<stdio.h>
#include<conio.h>
#define MAX 10

typedef struct node
{
int data;
struct node *next;
}node;
void init(node *hashtable[]);
void insert(node *hashtable[], int x);
node *find(node *hashtable[], int x);

node *hashtable[MAX]; //bucket table header

void main()
{
int n,x; node *f;
clrscr();

init(hashtable);
printf("1 for insert, 2 for find");
```

```
printf("Enter the choice");
scanf("%d",&n);


while(n<3)
{
switch(n)
{
        case 1:
                printf("Enter the inserting value\n");
                scanf("%d",&x);
                insert(hashtable, x);
                break;
        case 2:
            printf("Enter the searching value\n");
            scanf("%d",&x);
            f=find(hashtable, x);
            printf("Search element=%d\n",*f);
            break;
}
printf("Enter the choice\n");
scanf("%d",&n);
}
}

void init(node *hashtable[])
{
int i;
for(i=0;i<MAX; i++)
{
        hashtable[i]=NULL;
}
}

void insert(node *hashtable[], int x)
{
int loc; node *p, *q;
loc=x%MAX; // list to which the element mapped
printf("Location of inserting elementis=%d\n",loc);
p=hashtable[loc];
while(p!=NULL)
{
        p=p->next;
}
if(p==NULL); // inset the element from the fornt
{
        q=(node*)malloc(sizeof(node));
        q->data=x;
        q->next=NULL;
        q->next=hashtable[loc];
        hashtable[loc]=q;
```

```
}
}

node *find(node *hashtable[], int x)
{
int loc, count=0;
node *p;
loc=x%MAX;
p=hashtable[loc];
while(p!=NULL && x!=p->data)
{
        p=p->next;
        count++;
}
printf("Location of deleting Element=%d\n",count);
return(p);
}
```

Explanation:

- **Hash Table is implemented using an array of size MAX**.
- **Hash function** is key % MAX (Modulo division method).
- **Insertion function** places values at their computed index, handling collisions with **Linear Probing**.
- **Search function** checks if a value is present in the Hash Table.
- **Switch-case dynamically manages user operations**.

Sample Output:

mathematica
CopyEdit

```
Menu:
1. Insert
2. Search
3. Display Hash Table
4. Exit
Enter your choice: 1
Enter value to insert: 25
25 inserted at index 5.

Enter your choice: 1
Enter value to insert: 35
Collision at index 5! Using Linear Probing...
35 inserted at index 6.

Enter your choice: 3
Hash Table:
Index 0 -> Empty
Index 1 -> Empty
Index 2 -> Empty
Index 3 -> Empty
Index 4 -> Empty
Index 5 -> 25
Index 6 -> 35
Index 7 -> Empty
```

```
Index 8 -> Empty
Index 9 -> Empty

Enter your choice: 2
Enter value to search: 35
35 found at index 6.

Enter your choice: 4
Exiting program...
```

Conclusion:

- **Implemented Searching using Hash Table with Linear Probing**.
- **Used switch-case** for structured and interactive implementation.
- **Handled collisions using Linear Probing**.

Experiment 11: Heap Sort

Aim:

To implement **Heap Sort** using a **Binary Heap** in C.

Learning Outcomes:

- Understand the **Heap Data Structure**.
- Implement **Heap Sort** using **Heapify and Build Heap operations**.
- Use **switch-case** for structured implementation.

Algorithm (Heap Sort):

1. **Build a Max-Heap** from the given array.
2. **Extract elements one by one** from the heap.
3. **Swap the root (max element) with the last element** and reduce heap size.
4. **Heapify** the root to maintain the heap property.
5. Repeat until all elements are sorted.

C Code (Heap Sort with Switch-Case):

```
#include <stdio.h>

// Function to heapify a subtree rooted at index i
void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

// Function to perform Heap Sort
void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
```

```
        heapify(arr, n, i);

    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

// Function to display the array
void display(int arr[], int n) {
    printf("Sorted Array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[100], n, choice;

    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    do {
        printf("\nMenu:\n1. Perform Heap Sort\n2. Display Sorted Array\n3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                heapSort(arr, n);
                printf("Heap Sort completed!\n");
                break;
            case 2:
                display(arr, n);
                break;
            case 3:
                printf("Exiting program...\n");
                break;
            default:
                printf("Invalid choice! Try again.\n");
```

```
        }
    } while (choice != 3);

    return 0;
}
```

Sample Output:

mathematica
CopyEdit
Enter number of elements: 5
Enter elements: 4 10 3 5 1

Menu:
1. Perform Heap Sort
2. Display Sorted Array
3. Exit
Enter your choice: 1
Heap Sort completed!

Enter your choice: 2
Sorted Array: 1 3 4 5 10

Enter your choice: 3
Exiting program...

Experiment 12: Sorting Algorithms (Quick Sort, Bubble Sort, Insertion Sort, Merge Sort)

Aim:

To implement the following **sorting algorithms** using **switch-case** in C:

1. **Quick Sort**
2. **Bubble Sort**
3. **Insertion Sort**
4. **Merge Sort**

Learning Outcomes:

- Understand **various sorting algorithms** and their complexity.
- Implement **Quick Sort, Bubble Sort, Insertion Sort, and Merge Sort**.
- Use **switch-case** to select the sorting algorithm dynamically.

Algorithm (Sorting Methods):

- ***Quick Sort:***
  1. Select a **pivot**.
  2. Partition the array such that **elements smaller than pivot are left, larger are right**.
  3. Recursively sort the left and right subarrays.
- ***Bubble Sort:***

1. Compare adjacent elements and **swap if out of order**.
2. Repeat for the entire array until sorted.

- ***Insertion Sort:***

1. Pick an element and **place it in the correct position**.
2. Repeat for all elements.

- ***Merge Sort:***
  1. Recursively **divide the array into halves** until single elements.
  2. **Merge** sorted halves back into a single sorted array.

C Code (Sorting Algorithms with Switch-Case):

```
#include <stdio.h>

// Function for Quick Sort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = arr[high], i = low - 1;
        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
```

```
        int temp = arr[i + 1];
        arr[i + 1] = arr[high];
        arr[high] = temp;
        int pi = i + 1;

        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function for Bubble Sort
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
}

// Function for Insertion Sort
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i], j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Function for Merge Sort
void merge(int arr[], int l, int m, int r) {
    int i, j, k, n1 = m - l + 1, n2 = r - m;
    int L[n1], R[n2];

    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
```

```
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

// Function to display the array
void display(int arr[], int n) {
    printf("Sorted Array: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[100], n, choice;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    do {
        printf("\n1. Quick Sort\n2. Bubble Sort\n3. Insertion Sort\n4. Merge Sort\n5. Exit\n");
        scanf("%d", &choice);

        switch (choice) {
            case 1: quickSort(arr, 0, n - 1); break;
            case 2: bubbleSort(arr, n); break;
            case 3: insertionSort(arr, n); break;
            case 4: mergeSort(arr, 0, n - 1); break;
            case 5: return 0;
        }
        display(arr, n);
    } while (1);
}
```

Experiment 13: Stack Using an Array and Linked List

Aim:
To implement **stack operations** using **both an array and a linked list** in C.

Operations Implemented:
- **Push** (Insert element)
- **Pop** (Remove element)
- **Display Stack**

Algorithm:
- ***Stack Using an Array:***

1. Initialize top = -1 to represent an empty stack.
2. **Push**: Increment top and insert the element.
3. **Pop**: Remove the element from top and decrement top.
4. **Display**: Print stack elements from top to 0.

- ***Stack Using a Linked List:***

1. Define a struct Node containing data and a next pointer.
2. **Push**: Create a new node, link it to the previous top, and update the top pointer.
3. **Pop**: Remove the top node and update the top pointer.
4. **Display**: Traverse and print all nodes.

C Code (Stack Using an Array and Linked List with Switch-Case):
```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5  // Stack size

int stack[MAX], top = -1;

// Structure for stack using linked list
struct Node {
   int data;
   struct Node* next;
};

struct Node* topLL = NULL;

// Stack using an Array
void pushArray(int value) {
   if (top == MAX - 1) {
      printf("Stack Overflow!\n");
      return;
   }
   stack[++top] = value;
```

```
    printf("%d pushed to stack (Array).\n", value);
}

void popArray() {
    if (top == -1) {
        printf("Stack Underflow!\n");
        return;
    }
    printf("%d popped from stack (Array).\n", stack[top--]);
}

void displayArray() {
    if (top == -1) {
        printf("Stack is empty.\n");
        return;
    }
    printf("Stack (Array): ");
    for (int i = top; i >= 0; i--)
        printf("%d ", stack[i]);
    printf("\n");
}

// Stack using a Linked List
void pushLinkedList(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = topLL;
    topLL = newNode;
    printf("%d pushed to stack (Linked List).\n", value);
}

void popLinkedList() {
    if (topLL == NULL) {
        printf("Stack Underflow!\n");
        return;
    }
    struct Node* temp = topLL;
    printf("%d popped from stack (Linked List).\n", topLL->data);
    topLL = topLL->next;
    free(temp);
}

void displayLinkedList() {
    if (topLL == NULL) {
        printf("Stack is empty.\n");
        return;
    }
    struct Node* temp = topLL;
    printf("Stack (Linked List): ");
    while (temp) {
        printf("%d ", temp->data);
```

```
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int choice, value;

    do {
        printf("\nMenu:\n");
        printf("1. Push (Array)\n2. Pop (Array)\n3. Display (Array)\n");
        printf("4. Push (Linked List)\n5. Pop (Linked List)\n6. Display (Linked List)\n7. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &value);
                pushArray(value);
                break;
            case 2:
                popArray();
                break;
            case 3:
                displayArray();
                break;
            case 4:
                printf("Enter value to push: ");
                scanf("%d", &value);
                pushLinkedList(value);
                break;
            case 5:
                popLinkedList();
                break;
            case 6:
                displayLinkedList();
                break;
            case 7:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice! Try again.\n");
        }
    } while (choice != 7);

    return 0;
}
```

Experiment 14: Graph Representation Using an Array

Aim:
To implement **Graph Representation** using **Adjacency Matrix and Adjacency List** in C.

Operations Implemented:
- **Create Graph using Adjacency Matrix**
- **Display Graph**

Algorithm:
- ***Adjacency Matrix Representation:***
  1. Define a 2D array to store edges.
  2. Take number of vertices and edges as input.
  3. Update the adjacency matrix for each edge.
  4. Display the graph using matrix representation.

C Code (Graph Representation Using an Array with Switch-Case):
```
#include <stdio.h>

#define MAX 10

int adjMatrix[MAX][MAX] = {0};  // Initialize graph with no edges
int vertices, edges;

// Function to create a graph
void createGraph() {
    int src, dest;
    printf("Enter number of vertices: ");
    scanf("%d", &vertices);
    printf("Enter number of edges: ");
    scanf("%d", &edges);

    for (int i = 0; i < edges; i++) {
        printf("Enter edge (source destination): ");
        scanf("%d %d", &src, &dest);
        adjMatrix[src][dest] = 1;
        adjMatrix[dest][src] = 1;  // For an undirected graph
    }
}

// Function to display the graph
void displayGraph() {
    printf("Adjacency Matrix:\n");
    for (int i = 0; i < vertices; i++) {
```

```
        for (int j = 0; j < vertices; j++)
            printf("%d ", adjMatrix[i][j]);
        printf("\n");
    }
}

int main() {
    int choice;

    do {
        printf("\nMenu:\n");
        printf("1. Create Graph\n2. Display Graph\n3. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                createGraph();
                break;
            case 2:
                displayGraph();
                break;
            case 3:
                printf("Exiting...\n");
                break;
            default:
                printf("Invalid choice! Try again.\n");
        }
    } while (choice != 3);

    return 0;
}
```

Experiment 15: Graph Traversal Using BFS & DFS

Aim:

To implement **Graph Traversal (BFS & DFS)** using an **Adjacency Matrix** in C.

C Code (BFS & DFS with Switch-Case):

```
#include <stdio.h>
#include <stdbool.h>
#define MAX 10
int adj[MAX][MAX], visited[MAX], vertices;

void bfs(int start) {
    int queue[MAX], front = 0, rear = 0;
    bool visited[MAX] = {false};
    printf("BFS Traversal: ");
    queue[rear++] = start;
    visited[start] = true;

    while (front < rear) {
        int node = queue[front++];
        printf("%d ", node);

        for (int i = 0; i < vertices; i++)
            if (adj[node][i] && !visited[i]) {
                queue[rear++] = i;
                visited[i] = true;
            }
    }
    printf("\n");
}

void dfs(int node) {
    if (visited[node]) return;
    printf("%d ", node);
    visited[node] = true;

    for (int i = 0; i < vertices; i++)
        if (adj[node][i]) dfs(i);
}

int main() {
    printf("Enter number of vertices: ");
    scanf("%d", &vertices);
    createGraph();
    printf("Enter starting vertex for BFS: ");
    int start;
    scanf("%d", &start);
    bfs(start);
    printf("DFS Traversal: ");
    dfs(start);
    printf("\n");
    return 0;
}
```

Student Report Format

Galgotias University

Plot No.2, Sector-17A, Yamuna Expresway, Greater Noida, Gautam Buddh Nagar, U.P., India

Course Name:_____

Course Code: _____

School:_____ Year:_____
Semester:_____

Program:_____
Session:_____

Submitted By:	**Submitted To:**
Name:_____ _____ Admission No:_____ _____	_____ _____

Name	Semester	Section	Admission No.

Index

Sr. No.	Name of Experiment	Page No.	Date of Experiment	Signature of Faculty
1.	----------			
2.	----------			
3.	----------			
4.	----------			
5.	----------			
6.	----------			
7.	----------			
8.	----------			
9.	----------			
10.	----------			

Experiment No. 1

Objective/Aim: (Write the objective here)

Program: (Write Your program here)

Sample Run:

Show the sample output here